

생산라인 모니터링 채팅 프로그램

5팀

팀장: 배기현

팀원: 가대교, 소현섭, 박준혁, 윤영서

BUSINESS PROPOSAL

| 목차

Chapter 1. **업무 분배도**

Chapter 2. **프로젝트 개요**

Chapter 3. **프로젝트 수행 절차 및 방법**

Chapter 4. **시스템 구현 기술**

Chapter 5. **순서도**

Chapter 6. **코드 소개**

Chapter 7. **후기**

업무 분배도

훈련생	역할	담당 업무
배기현	팀장	바코드 검출 및 플라스크 서버 기능 구현
가대교	팀원	DB스키마 설계 및 테이블 생성
소현섭	팀원	소켓통신 기반 채팅 프로그램 구현
박준혁	팀원	비밀번호 해쉬화 및 라즈베리 센서 코드 구현
윤영서	팀원	MFC 기능 및 UI 기능 구현

프로젝트 개요

1

프로젝트 주제 및 선정 배경, 기획의도

생산 현장의 신속한 이상 대응을 위해 영상과 채팅을 통합한 실시간 모니터링 시스템을 구축하였다.

2

프로젝트 내용

Flask 기반으로 영상 스트리밍·채팅·바코드 인식을 통합한 실시간 생산라인 모니터링 시스템.

3

활용 장비 및 재료

웹캠과 라즈베리파이,온 습도 센서

4

활용방안 및 기대효과

업무 효율 향상, 품질 관리 강화, 문제 발생 시 대응 속도 개선.

프로젝트 수행 절차 및 방법

구분	기간	활동	비고
사전 기획	9/27~9/29	프로젝트 기획 및 주제 신청/기획안 작성	아이디어 선정 및 역할 분배
플라스크 서버 기능구현	9/30~10/14	플라스크 서버및 바코드 검출	
라즈베리파이 코드 작성	9/30~10/2	웹캠 구동 및 컨베이어 벨트 구동	
DB테이블 구현	9/30~10/1	채팅 내역 저장, 사용자 계정 생성 구현,온습도 저장	데이터 베이스 보안 강화
MFC기능 구현	9/30~10/15	실시간 영상, 온습도 표시, 컨베이어 벨트 제어,채팅	
총 개발기간	9일		

시스템 구현 기술

1 **mfc: 플라스크 서버와의 통신으로 데이터베이스 접근 하는 클라이언트**

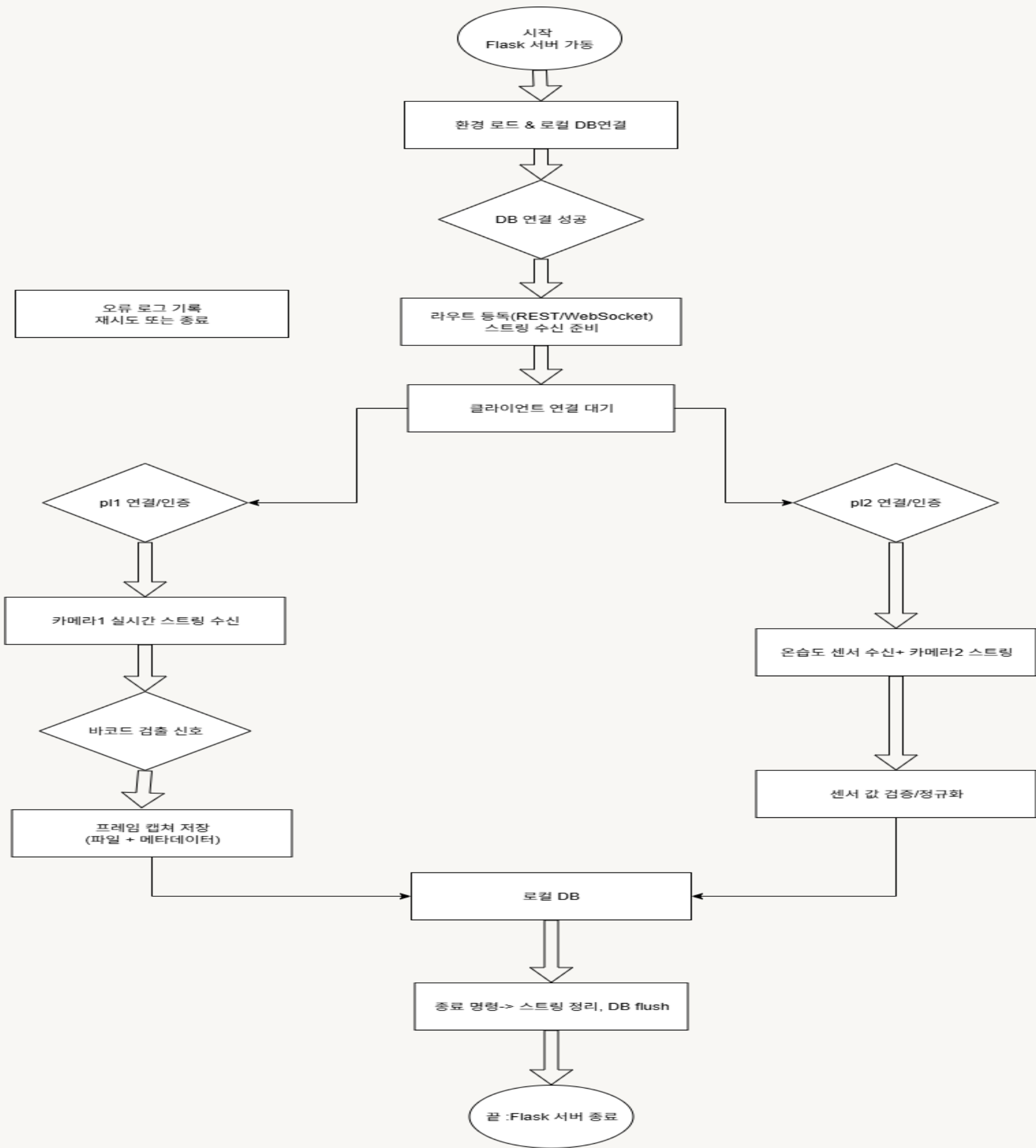
2 **플라스크 서버: 파이썬을 사용한 서버 역할**

3 **라즈베리파이: 컨베이어 벨트와 웹캠을 사용하여 불량 검출시스템 구축**

4 **mysql: 유저의 로그인 정보와 채팅내역 저장 역할**

Chapter 5.

순서도



```
• CREATE TABLE `user_account` (  
  `id` bigint NOT NULL AUTO_INCREMENT,  
  `emp_no` varchar(32) NOT NULL,  
  `password_hash` varchar(255) NOT NULL,  
  `role` enum('ADMIN','STAFF') NOT NULL,  
  `joined_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `is_active` tinyint(1) NOT NULL DEFAULT '1',  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_emp_no` (`emp_no`)  
) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

어카운트


```
• CREATE TABLE `barcode_detection_log` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `barcode` varchar(20) NOT NULL,  
  `product_name` varchar(100) DEFAULT NULL,  
  `confidence` decimal(5,2) DEFAULT NULL,  
  `image_path` varchar(255) DEFAULT NULL COMMENT '이미지 파일 경로',  
  `image_filename` varchar(100) DEFAULT NULL COMMENT '이미지 파일명',  
  `detected_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,  
  `bbox_x1` int DEFAULT NULL COMMENT '바운딩박스 좌상단 X',  
  `bbox_y1` int DEFAULT NULL COMMENT '바운딩박스 좌상단 Y',  
  `bbox_x2` int DEFAULT NULL COMMENT '바운딩박스 우하단 X',  
  `bbox_y2` int DEFAULT NULL COMMENT '바운딩박스 우하단 Y',  
  PRIMARY KEY (`id`),  
  KEY `idx_barcode` (`barcode`),  
  KEY `idx_detected_at` (`detected_at`)  
) ENGINE=InnoDB AUTO_INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

코드 소개

코드 소개

```

177 def conveyor_loop(motor):
178     """
179     상태(state.should_run)에 따라:
180     - True → (2.0s 전진 → 1.5s 정지) 루프 반복
181     - False → 완전 정지 유지
182     """
183     print(f"[Conveyor] 기본 루프: {FORWARD_SEC}s 전진 → {PAUSE_SEC}s 정지 (서버 상태로 ON/OFF)")
184     try:
185         while not stop_event.is_set():
186             if state.get_should_run():
187                 # 전진
188                 if motor:
189                     motor.forward()
190                 t0 = time.time()
191                 while not stop_event.is_set() and (time.time() - t0) < FORWARD_SEC:
192                     # 중간에 STOP 지시가 오면 즉시 정지
193                     if not state.get_should_run():
194                         break
195                     time.sleep(0.01)
196
197                 # 정지
198                 if motor:
199                     motor.stop()
200
201                 # STOP 지시가 왔으면 정지 상태 유지로 전환
202                 if not state.get_should_run():
203                     # 바로 다음 루프로 넘어가 False 분기에서 유지
204                     continue
205
206                 # 기본 정지 시간
207                 t1 = time.time()
208                 while not stop_event.is_set() and (time.time() - t1) < PAUSE_SEC:
209                     if not state.get_should_run():
210                         break
211                     time.sleep(0.01)
212             else:
213                 # 원하는 상태가 STOP이면 정지 유지
214                 if motor:
215                     motor.stop()
216                 time.sleep(0.05)
217     finally:
218         if motor:
219             try:
220                 motor.stop()
221                 motor.close()
222             except Exception:
223                 pass
224     print("[Conveyor] 루프 종료")

```

비밀번호 해쉬화 코드

```
# 비밀번호 해쉬화 코드
import bcrypt, unicodedata

pw = unicodedata.normalize("NFC", "배기현") # 정규화(권장)
hash_ = bcrypt.hashpw(pw.encode("utf-8"), bcrypt.gensalt(12)).decode()
print(hash_)

pw_2 = unicodedata.normalize("NFC", "윤영서") # 정규화(권장)
hash2_ = bcrypt.hashpw(pw_2.encode("utf-8"), bcrypt.gensalt(12)).decode()
print(hash2_)

pw_3 = unicodedata.normalize("NFC", "박준혁") # 정규화(권장)
hash3_ = bcrypt.hashpw(pw_3.encode("utf-8"), bcrypt.gensalt(12)).decode()
print(hash3_)

pw_4 = unicodedata.normalize("NFC", "소현섭") # 정규화(권장)
hash4_ = bcrypt.hashpw(pw_4.encode("utf-8"), bcrypt.gensalt(12)).decode()
print(hash4_)

pw_5 = unicodedata.normalize("NFC", "가대교") # 정규화(권장)
hash5_ = bcrypt.hashpw(pw_5.encode("utf-8"), bcrypt.gensalt(12)).decode()
print(hash5_)
|
```



```
def detect_balanced_barcode(frame_data, broadcast_fn=None):
    """ 🌀 Balance 바코드 검출 (등록 제품 우선 선택 + 이미지 저장) """
    global last_barcode_detection_time, last_detected_barcode, barcode_detection_count, rejected_barcode_count

    if not BARCODE_DETECTION_AVAILABLE:
        return []

    try:
        current_time = time.time()

        # 간격 체크
        if current_time - last_barcode_detection_time < BARCODE_DETECTION_INTERVAL:
            return []

        # JPEG 디코딩
        nparr = np.frombuffer(frame_data, np.uint8)
        frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        if frame is None:
            return []

        # 그레이스케일 변환
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Balance 검출
        barcodes = pyzbar.decode(gray)

        # 바코드가 없으면 약간의 전처리 후 재시도
        if not barcodes:
            enhanced = cv2.convertScaleAbs(gray, alpha=1.1, beta=10)
            barcodes = pyzbar.decode(enhanced)

        # 바코드가 하나도 없으면 조용히 종료
        if not barcodes:
            return []
```

바코드 검증

```
def validate_barcode_balanced(barcode_data):
    """🔍 Balance 바코드 패턴 검증 (정확하지만 너무 엄격하지 않게)"""
    try:
        # 1. 기본 길이 검증 (10~15자리 허용)
        if len(barcode_data) < 10 or len(barcode_data) > 15:
            return False, "길이_오류"

        # 2. 숫자만 포함 검증
        if not barcode_data.isdigit():
            return False, "숫자_오류"

        # 3. 등록된 Bamgohsi 제품 우선 허용
        if barcode_data in BARCODE_PRODUCT_MAP:
            return True, "등록_제품"

        # 4. 명백한 잘못된 패턴만 차단 (Balance)
        obvious_invalid_patterns = [
            r"^0{8,}",          # 00000000...
            r"^1{8,}",          # 11111111...
            r"^[0-2]{13}$",     # 000~222
            r"^9{8,}",          # 99999999...
        ]

        for pattern in obvious_invalid_patterns:
            if re.match(pattern, barcode_data):
                return False, "명백한_잘못된_패턴"

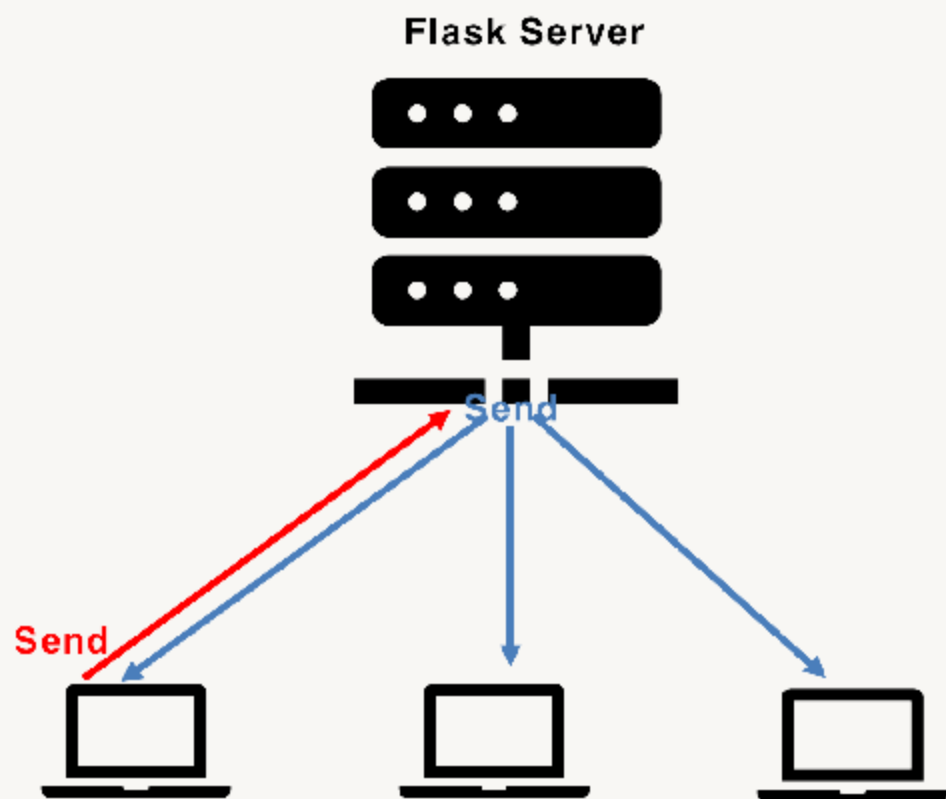
        # 5. 일반적인 바코드 패턴 허용 (Balance)
        if len(barcode_data) == 13:
            return True, "표준_13자리_바코드"

        if len(barcode_data) == 12:
            return True, "표준_12자리_바코드"

        return True, "일반_바코드"

    except Exception as e:
        return False, "검증_오류"
```

소켓 통신기반 채팅(TCP)



클라이언트들의 접속을 대기 합니다.

```
start_tcp_server()
```

서버에 접속하는 클라이언트들의 메시지를 받고 보내기 위한 소켓을 연결하는 쓰레드를 실행합니다.

로그인 처리를 하며 중복로그인을 방지합니다. 이미 접속중인 클라이언트가 있다면 접속을 종료시킵니다.

인증에 성공하면 클라이언트 정보를 clients 리스트에 추가해 관리합니다.

```
verify_user(emp_no: str, password: str)
```

로그인에 성공한 클라이언트들이 채팅을 보내면 이를 수신합니다.

채팅 내역을 데이터 베이스에 저장합니다.

Boardcast함수로 서버에 접속해 있는 채팅을 보낸 클라이언트를 제외한 모든 클라이언트들에게 메시지를 전달합니다.

```
handle_client(client_socket: socket.socket, addr)
```

```
broadcast(message_bytes: bytes, sender_sock: socket.socket)
```

```
void CMonitoringProgramView::OnBnClickedChatButton()
{
    CEdit* pChatEdit = (CEdit*)GetDlgItem(IDC_CHAT_EDIT);
    CString buffer;
    pChatEdit->GetWindowText(buffer);
    buffer.Trim();

    if (buffer.IsEmpty()) return;

    if (!m_chatClient.IsConnected()) {
        AfxMessageBox(L"서버에 연결되어 있지 않습니다.");
        return;
    }

    // 메시지 200자 이내로 제한
    if (buffer.GetLength() > 200)
    {
        AfxMessageBox(_T("메세지는 200자 이내로 입력해주세요."));
        return;
    }

    if (m_chatClient.SendMessage(buffer)) {
        CString selfLine;
        selfLine.Format(L"%s > %s", m_empNo.GetString(), buffer.GetString());
        AddStringToListBoxWithWrap(&m_myListBox, selfLine);
    }
    else {
        AddStringToListBoxWithWrap(&m_myListBox, L"++ 메시지 전송 실패 ++");
    }

    pChatEdit->SetWindowText(L"");
    pChatEdit->SetFocus();
}
```


Chapter 6.

코드 소개

MFC

```
void CMonitoringProgramView::AddStringToListBoxWithWrap(CListBox* pListBox, const CString& text)
{
    CRect rect;
    pListBox->GetClientRect(&rect); // 리스트박스 폭
    int maxWidth = rect.Width() - 4; // 약간 여유

    CDC* pDC = pListBox->GetDC();
    CFont* pOldFont = pDC->SelectObject(&m_Font); // 현재 사용하는 폰트 설정

    CString line;
    int curWidth = 0;

    for (int i = 0; i < text.GetLength(); i++)
    {
        CString ch = text.Mid(i, 1);
        CSize sz = pDC->GetTextExtent(ch);

        if (curWidth + sz.cx > maxWidth)
        {
            pListBox->AddString(line); // 줄 추가
            line = ch;
            curWidth = sz.cx;
        }
        else
        {
            line += ch;
            curWidth += sz.cx;
        }
    }
}
```

```
if (!line.IsEmpty())
    pListBox->AddString(line);

pDC->SelectObject(pOldFont);
pListBox->ReleaseDC(pDC);

// 자동 스크롤
const int count = pListBox->GetCount();
if (count > 0) {
    pListBox->SetTopIndex(count - 1);
}
```




소현섭: 서버와의 연동이 어려웠습니다.
실시간 채팅을 만들면서 데이터 처리를 어디서
해야 하는지 고민한 게 큰 기억에 남았습니다.
통신에 대한 이해가 느는 좋은 기회가 됐습니다.

박준혁: 안써봤던 기술과 배워던 기술을
써서 선행학습과 복습을 할 수 있던
좋은 기회였습니다.

윤영서: 협업 중 코드 손실을 겪으며 버전 관리의
중요성을 느꼈고, MFC의 한계로 원하는 기능 구
현에 어려움을 겪으며 공부의 필요하다는 것을
깨달았습니다.

가대교: 데이터베이스를 맡으면서 수업때
배웠던 내용을 복습했고, 어려움이 있지만
팀원들과 같이 했고 이를 통해서 팀원들이
소중하다고 느꼈습니다.

배기현: 팀장으로서 제 부족한 부분들에 대해 돌
아보게 되는 프로젝트 였습니다. 부족함을 메워
가고 앞으로 잘 해봐야겠다 생각이 들었습니다.

발표를 들어주셔서
감사합니다.

THANK YOU
